

JAVA 8 TO 21 NEW FEATURES

1. LAMBDA & FUNCTIONAL INTERFACE & STREAMS INTRO

INTRO

Interviewer
Explain Lambda and Functional Interface?
Why Streams?

Candidate

Functional Interface:
an interface with a
single abstract method

Functional Interface: an interface
with a single abstract method

Examples: Runnable, Predicate, Function,
Supplier, Consumer

Use @FunctionalInterface annotation

Lambda: concise way to implement it: {} -> {}

BEFORE: Java 7 (Anonymous class) → AFTER: Java 8 (Lambda)

```
Runnable r = new Runnable() {
    · public void run() {
    · System.out.println("Hello");
};
```

— verbose, lots of boilerplate

→ after Java 8



```
Runnable r = () -> System.out.println("Hello");
```

— concise, clean, one line

After: Stream API
Before: for-loop (imperative)

```
List<Integer> result =
new ArrayList<>();

for (int x : list) {
    · if (x > 10) {
    · result.add(x * 2);
    }
}
```

— manual loop, more code



After: Java 8 Stream

STREAMS: Declarative Data Processing

Streams provide a declarative, pipeline-style approach to
processing collections, like SQL queries.

Key idea: lazy intermediate operations, eager terminal operation

Common Functional Interfaces:

Predicate<T>
boolean test(T t)

Function<T, R>
R apply(T t)

Consumer<T>
void accept(T t)

Source

filter()

map()

collect()

list / collection

intermediate (lazy)

intermediate (lazy)

terminal (eager)

Flow: Source → filter → map → collect — operations run only when terminal is called

✓ Benefits:

✓ More readable and concise

✓ Supports parallel processing with .parallelStream()

✓ Cleaner, less boilerplate, functional style

JAVA 8 TO 21 NEW FEATURES 2/20

2. STREAM API DEEP - INTERMEDIATE vs TERMINAL

Interviewer (red):

- Intermediate vs Terminal operations?
- flatMap vs map?

Candidate (blue):

- Intermediate — lazy; returns Stream; not executed until terminal
e.g. filter • map • flatMap • distinct • sorted • peek • limit
- Terminal — eager; triggers execution; returns result
e.g. collect • forEach • count • reduce • min • max • anyMatch • allMatch • findFirst

map → one-to-one transform

Transforms each element to one element.

```
Stream<String> lengths =  
names.stream()  
.map(s -> s.length());
```

"Alice" → 5, "Bob" → 3, "Eve" → 3

["Alice", "Bob", "Eve"] → [5, 3, 3]
map(s -> s.length())

flatMap → one-to-many → flatten

Flattens nested structures into single Stream.

```
List<List<Intedep>> Lists = ...  
Lists.stream()  
.flatMap(Collection::stream)
```

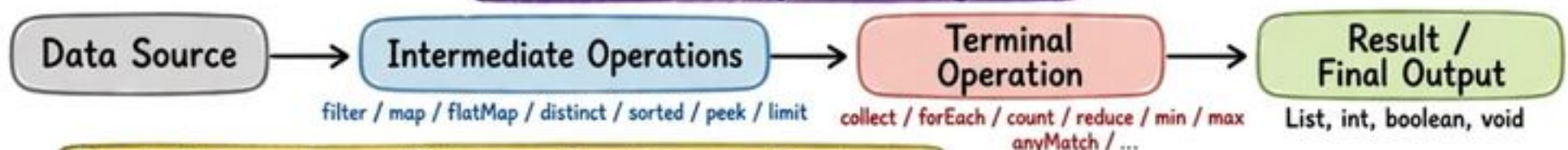
[[1,2][3,4,5]] → [1,2,3,4,5]

[[1,2], [3,4,5]] ↓ flatMap(Collection::stream)
[1, 2, 3, 4, 5]

Intermediate vs Terminal – Quick Comparison Table

Aspect	Intermediate	Terminal
1. Behavior	Lazy — no execution	Eager — triggers execution
2. Return Type	Returns a Stream	Returns non-Stream result
3. Examples	filter(), map(), sorted(), limit()	collect(), forEach(), count(), reduce()

Stream Lifecycle Diagram



Real Example – Flatten Employees Projects

```
employees.stream()  
.flatMap(e -> e.getProjects().stream())  
.distinct(System.out::println);  
// Streams all projects from each employee into a single stream of projects
```


JAVA 8 TO 21 NEW FEATURES

3/20

3.

3. OPTIONAL & DATE TIME API - JAVA 8

Interviewer

"Why Optional? Java 8 Date Time API vs old Date?"

Candidate response:

OPTIONAL - AVOID NULL POINTER

Explain: Optional avoids NullPointerException
— wrapper for nullable value.

Before: messy null check —
NPE risk pyramid of doom



```
user != null  
user.getAddress() != null  
user.getAddress().getCity() != null
```

→ chained if-statements
→ NPE risk!

After clean:

```
Optional<User> opt = Optional.ofNullable(user);  
opt.map(User::getAddress)  
  .map(Address::getCity)  
  .orElse("Unknown"); // returns "Unknown" if empty
```

Methods:

- of() — creates Optional with non-null value
- ofNullable() — creates Optional, allows null
- isPresent() — checks if value is present
- orElse() — provides default if empty
- orElseGet() — provides default via supplier
- orElseThrow() — throws if empty
- map() — transform value if present
- flatMap() — chain Optionals together

⚠ Warning: Never use get() directly!

DATE TIME API - java.time

Before: Date / Calendar / SimpleDateFormat

- ✗ NOT thread-safe
- ✗ Mutable — can be changed
- ✗ 0-indexed months = bug (Jan = 0)
- ✗ Error-prone, not thread-safe formatting

After: java.time package

- ✓ Immutable & thread-safe
- ✓ Clear, modern, intuitive API
- ✓ 1-indexed months (Jan = 1)

Key Classes:

- ✓ LocalDate — date only (e.g. 2024-08-05)
- ✓ LocalTime — time only (e.g. 14:30:00)
- ✓ LocalDateTime — date + time
- ✓ ZonedDateTime — date + time + time zone
- ✓ Instant — timestamp instant in UTC
- ✓ Duration — time duration (hours, minutes, seconds)
- ✓ Period — date period (years, months, days)
- ✓ DateTimeFormatter — thread-safe formatter

Example:

```
LocalDate today = LocalDate.now();  
LocalDate nextWeek = today.plusDays(7); // +7 days from today
```

Compare: OLD Date/Calendar vs NEW java.time

	OLD Date/Calendar	NEW java.time
Thread Safe	✗ Not thread-safe (mutable)	✓ Immutable & thread-safe
Month Index	✗ 0-indexed months (Jan = 0)	✓ 1-indexed months (Jan = 1)
Formatting	✗ SimpleDateFormat not thread-safe	✓ DateTimeFormatter is thread-safe
Design	✗ Cumbersome, legacy	✓ Modern, fluent API

JAVA 8 TO 21 NEW FEATURES

4. COMPLETABLEFUTURE & DEFAULT STATIC METHODS

4/20

4

Interviewer:
CompletableFuture vs Future?
Default methods in interface?

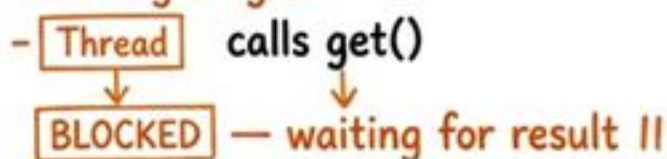
Candidate:
Future vs CompletableFuture
explanation.

1. FUTURE vs COMPLETABLEFUTURE

Future

- blocking `get()` — thread waits
- no chaining
- manual, synchronous

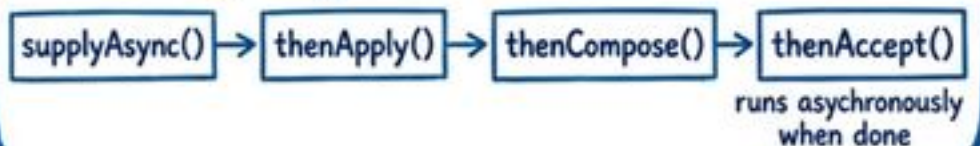
Blocking diagram:



CompletableFuture — Java 8

- non-blocking, async
- supports chaining / callbacks

Chaining diagram:



2. COMPLETABLEFUTURE METHODS LIST:

- `supplyAsync()` — async with return value
- `runAsync()` — async, no return
- `thenApply()` — transform result
- `thenCompose()` — flatMap for futures

- `allOf()` — wait for all futures to complete
 - `anyOf()` — wait for any future to complete
- `.thenAccept(System.out::println);`
→ Runs async, chains operations, prints result when ready

default methods

- Allows adding new methods without breaking implementations
- Examples in List interface:
 - `forEach()`
 - `removeIf()`
 - `splitterator()`

Syntax:

```
interface MyInterface {  
    • default void foo() {  
        System.out.println("default");  
    }  
    static void bar() { ... }  
}
```

- Static methods act as utilities — e.g. `Comparator.comparing()`
- Diamond problem is resolved by overriding the method in the implementation class

JAVA 8 TO 21 NEW FEATURES

5/20

5. JAVA 9 TO 11 - MODULES VAR COLLECTORS & HTTP CLIENT

Interviewer (red):

- Java 9 Modules?
- var?
- Java 11 features?

Candidate (blue):

- Java 9: Module System (JPMS)

- `module-info.java` →

```
module myapp {  
    requires java.sql;  
} exports com.myapp;
```
- encapsulation → strong modular JDK (Jigsaw)
- jshell → REPL for quick testing
- Factory methods → `List.of()` / `Set.of()` / `Map.of()`
→ creates immutable collections
- Stream improvements → `takeWhile()` / `dropWhile()`
- Private methods now allowed in interfaces

JAVA 10 → var (Local Variable Type Inference)

- `var` → local variable type inference
- `var list = new ArrayList-String());`
- `var` is NOT a keyword — only for local variables
(type is inferred at compile time, used for locals only)

var example:

```
var message = "Hello Java 10"; // inferred as String  
var numbers = List.of(1, 2, 3); // inferred as List-String>
```

JAVA 11 → New Features

- String new methods:
 - `isBlank()` → returns true if string is empty or blank
 - `lines()` → returns `Stream-String>` of lines
 - `strip()` → trims whitespace (Unicode-aware, improved trim)
 - `repeat(n)` → repeats the string n times
- `var` in lambda expression:
`(var x, var y) -> x + y`
- Files API updates:
`String data = Files.readString(Paths.get("file.txt"));`
`Files.writeString(Paths.get("output.txt"), data);`

JAVA 11 → New HTTP Client (`java.net.http`)

- `HttpClient` → modern API, supports sync & async, HTTP/2 support
- replaces old `HttpURLConnection`

```
HttpClient client = HttpClient.newHttpClient();  
HttpRequest request = HttpRequest.newBuilder()  
    .uri(URI.create("https://api.example.com"))  
    .GET();
```

`List.of()` vs old `Arrays.asList()`

Old: `Arrays.asList()` → mutable (but fixed-size)

```
List-String> list = Arrays.asList("a", "b");  
list.add("c"); // allowed but throws at runtime if fixed size
```

New: `List.of()` → immutable / cleaner

```
List-String> list = List.of("a", "b");  
list.add("c"); // throws UnsupportedOperationException
```


JAVA 8 TO 21 NEW FEATURES

6/20
6

JAVA 12 TO 14 - SWITCH EXPRESSIONS & TEXT BLOCKS

Interviewer:
Switch Expressions?
Text Blocks?

Candidate: Java 12 Switch
Expressions → no fall-through,
arrow → returns value.

Switch Expressions (Java 12-14)

No break bug ☒ — now uses → arrow and returns a value directly

BEFORE (Java 8-11)

```
switch(day) {  
  case MON:  
    result = 1;  
    break;  
}
```

→ requires break to avoid fall-through

AFTER (Java 12+)

```
var result = switch(day) {  
  case MON, TUE → 1;  
  case WED → {  
    yield 2;  
  }  
  default → 0;  
}
```

→ no break needed, returns value directly, multi-case with ,

BEFORE (Messy)

```
String json = "{  
  \"name\": \"John\"  
}  
→ lots of  
  and \" escapes, hard to read
```

AFTER (Java 13+)

```
String json = """ {  
  \"name\": \"John\"  
} """,
```

→ clean, preserves format, no escaping needed

Java 14 Improvements

- Helpful NullPointerException — JVM now tells you exactly which variable is null:
Exception: Cannot invoke "user.getName()" because "user" is null
- Records (Preview) — ^{record} x, int y) {; // auto getters, equals, hashCode, toString

JAVA 8 TO 21 NEW FEATURES

7/20

7

7. RECORDS - JAVA 14 / 16

Interviewer

What are Records?
Why use?

Before — POJO (50 lines)

```
public class User {  
    private final String name;  
    private final int age;  
    public User(String name, int age) {  
        < this.name = name;  
        < this.age = age;  
    }  
    public String getName() { return name; }  
    public int getAge() { return age; }  
}
```

- ~50 lines of boilerplate ... }
- Manual, repetitive, error-prone

Candidate

Records → compact, immutable data carrier class that replaces POJO boilerplate.

- Before: class User { private final String name; private final int age; constructor • getters • equals • hashCode • toString • all manual — about 50 lines }
- After: record User(String name, int age) { }
— one line, auto-generated constructor, getters, equals(), hashCode(), toString()
- ✓ Immutable • thread-safe
- Can add validation with a compact constructor:
record User(String name, int age) { if (age < 0)
throw new IllegalArgumentException("age cannot be negative"), }

After — Record (1 line)

```
record User(String name, int age) {
```

- 1 line only
- Auto-generated: constructor, getters, equals(), hashCode(), toString()
- Concise, immutable, clear

Record Components Diagram — What the compiler generates for you

Components

record User(String name, int age)
The declared record components

compiler
generates

Auto Fields

```
• private final String name;  
• private final int age;  
fields are private, final, immutable
```

→ auto
methods →

Accessors & Methods

- Constructor: User(String name, int age)
- Getters: name(), age()
- equals(), hashCode(), toString()

Note: Records are immutable by design. Fields are final. They cannot extend another class.
You can still implement interfaces and add static methods or fields.

JAVA 8 TO 21 NEW FEATURES

8/20
big 8

8. SEALED CLASSES
& PATTERN MATCHING
INSTANCEOF

Interviewer (red)
"Sealed Classes?
Pattern Matching
for instanceof?"

Candidate (blue)
Here is the breakdown for
interviews:
Candidate section:

• Sealed Classes —

Java 15 preview, Java 17 final —
restrict which classes can extend.

- Restricts which classes can extend; controlled hierarchy.

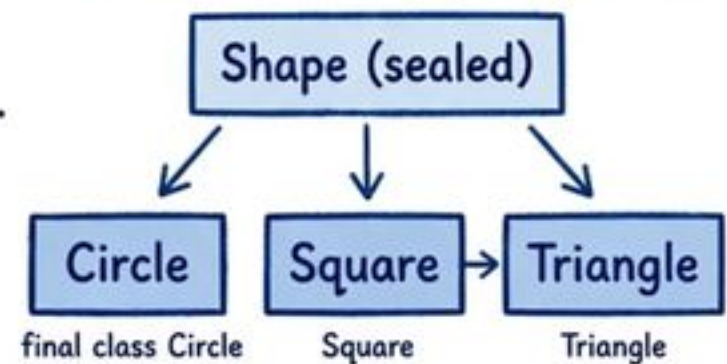
Syntax:

```
sealed class Shape permits Circle, Square, Triangle {}  
final class Circle extends Shape {}
```

Benefits:

- Controlled hierarchy — only permitted subtypes can extend
- Enables exhaustive switch — compiler knows all subtypes
- Safer design — improves maintainability

Sealed hierarchy diagram:



- ## • Pattern Matching instanceof —
- Java 14 preview, Java 16 final
avoids explicit cast.
- Bind the variable directly in the instanceof check; no cast needed.

Before (old):

```
if (obj instanceof String) {  
    String s = (String) obj; // cast required  
    int len = s.length();  
    • Requires explicit cast to String
```

After (Java 16+):

```
if (obj instanceof String s) {  
    int len = s.length(); // no cast needed  
} • Variable 's' is available, no cast needed
```

Combine with logic:

```
if (obj instanceof String s) && s.length() > 5) {  
    System.out.println(s + " is long, length=" + s.length());  
} • 's' is in scope only when the condition is true
```


JAVA 8 TO 21 NEW FEATURES

9/20
9.

9. JAVA 17 - SWITCH PATTERN MATCHING PREVIEW & NEW FEATURES

Interviewer

Pattern Matching for switch?
Java 17 LTS features?

Candidate

Pattern Matching for switch –
Java 17 preview,
Java 21 final, simplifies type
checking.

– Simplification →

BEFORE – intosdued chain:

```
if (shape intosdued Circle c) { ... }  
else if (shape intosdued Square s) { ... }  
else if (shape intosdued Triangle t) { ... }
```

Messy, vorbose



AFTER – Switch Pattern Matching Java 17 preview:

```
switch(shape) {  
- case Circle c → System.out.println("Circle radius  
  " + c.radius());  
- case Square s → System.out.println("Side " + s.side());  
  case null → System.out.println("Null!");  
- case default → System.out.println("Unknown");  
} Handles null, guards ✓
```

Guard with when clause:

```
case Circle c when c.radius() > 10 →  
  System.out.println("Large Circle");
```

JAVA 17 LTS FEATURES - WHY JAVA 17 = NEW JAVA 8:

- Sealed Classes final (JEP 409)
- Records final (JEP 395)
- Compact Number Format
- New macOS rendering pipeline
- Strongly Encapsulate JDK Internals (JEP 403)
- Removed RMI Activation

Java 17 LTS = long term support, new Java 8

10. JAVA 21 - VIRTUAL THREADS — LOOM GAME CHANGER

Interviewer:

- What are Virtual Threads?
- Difference: Platform vs Virtual?
- Why Loom?

Candidate:

- Virtual Threads = Java 21 lightweight threads managed by JVM not OS
- Millions vs Platform Threads = 1:1 OS thread, heavy, limited to thousands

Before: Thread-per-Request Model (Traditional)

- Uses ExecutorService with 200 threads (Tomcat max)
- Each request = 1 Platform Thread blocking on I/O
- Threads are expensive → waste → limited to ~200-1000 requests
- Blocking I/O = thread waits idle, resources wasted

After: Virtual Thread (Java 21 / Loom)

- Thread.startVirtualThread(1) → handleRequest()
- Executors.newVirtualThreadPerTaskExecutor() → creates millions cheaply
- Blocking is OK! Does NOT pin carrier thread

- scales to millions of concurrent requests

Platform Thread

- 1:1 with OS Thread
- Heavy, ~1MB stack
- Limited: Thousands

OS Thread 1 OS Thread 2 OS Thread 3...
↓
OS Kernel

Virtual Thread

- Managed by JVM (Loom)
- Lightweight, tiny stack
- Millions in JVM

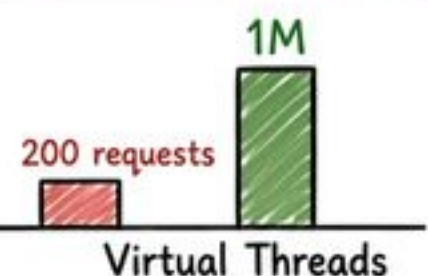
Carrier Thread (OS Platform Thread)

Virtual Thread 1...2...3... → mounted/unmounted
.....
millions

Throughput Comparison (I/O-bound workload)

Concurrent
Requests

Important: - Don't pool virtual threads! They are cheap to create/discard
- Use try-with-resources to auto-close executor



JAVA 8 TO 21 NEW FEATURES

11/20
11

11. JAVA 21 - SEQUENCED COLLECTIONS & STRING TEMPLATES



Interviewer:

Sequenced Collections?
String Templates?
Explain Java 21 features?

Candidate:

Here is the quick
interview-style
answer:

1. SEQUENCED COLLECTIONS —

Java 21 new interface for ordered collections with first • last • reversed.

Before:

- `List.get(0)` // get first element
- `list.get(list.size() - 1)` // get last element
- `LinkedHashSet`? No direct first/last access
- awkward

After:

- `sequencedCollection.getFirst()` // first element
- `getLast()` // last element
- `addFirst(e), addLast(e)` // add to start/end
- `reversed()` // reversed view

Implementations now: `List`, `LinkedHashSet`, `LinkedHashMap`, and `Deque` now implement `SequencedCollection` / `SequencedSet` / `SequencedMap`.

2. CODE COMPARISON:

Before

```
var first = list.get(0);  
var last = list.get(list.size() - 1);  
- verbose, messy, prone to injection
```

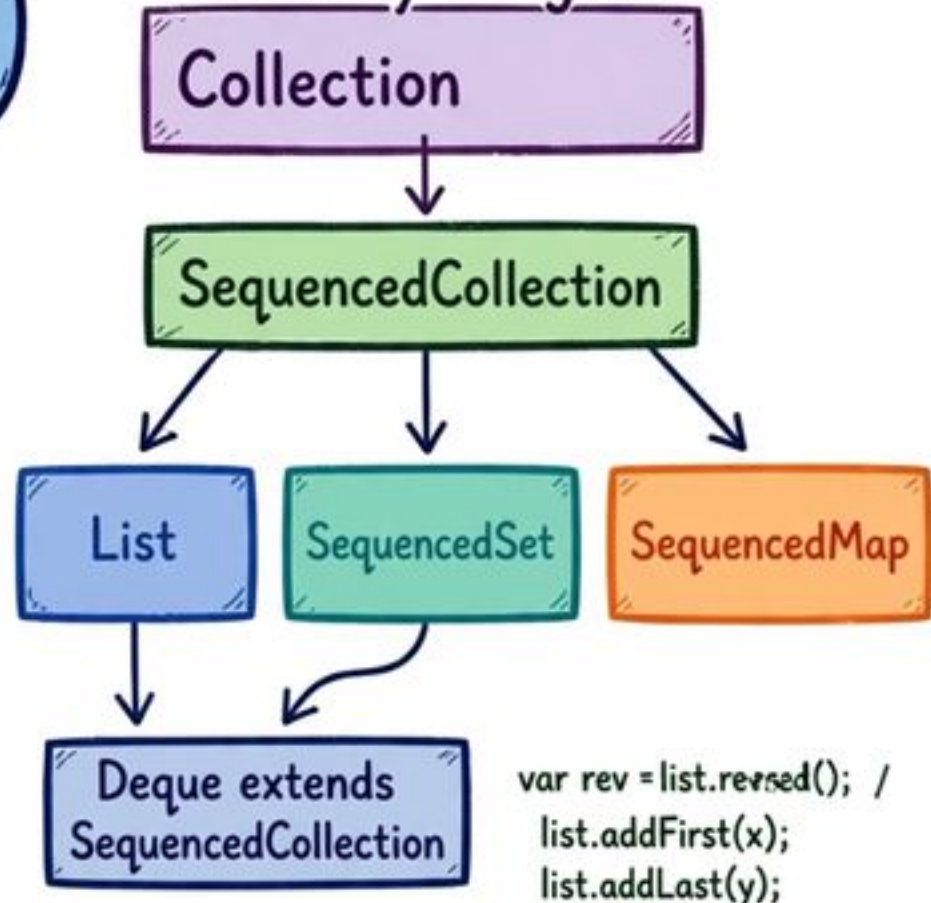
After

```
String s = STR."Hello \{name\} age \{age\}";  
- automatic safe interpolation  
String s2 = FMT."%02d\{age\} years";  
- %02d formats age with leading zeros
```

✓ Benefits:

- safe SQL / JSON construction
- clean, readable interpolation
- prevents injection vulnerabilities

Hierarchy Diagram:



3. STRING TEMPLATES —

Java 21 Preview: `STR.\"" for safe interpolation`

JAVA 8 TO 21 NEW FEATURES

12/20
12

12. JAVA 8 TO 21 - COLLECTORS & PARALLEL STREAMS TRICKY.

Interviewer:

Tricky Collectors?
parallelStream pitfalls?

Candidate:

answer structure.

Collectors:

- groupingBy: `list.stream().collect(groupingBy(Dept))`
- counting(), mapping(), partitioningBy(), teeing()
- Java 12: teeing() merges two collectors into one

Example:

```
Map<Dept, Long> countByDept =  
    . 'list.stream()  
      .collect(groupingBy(  
        Employee::getDept,  
        counting());    ));
```

→ Groups employees by Department
→ returns count per Dept

parallelStream():

Uses `ForkJoinPool` common pool for parallel processing
— fast but pitfalls:

- ⚠ Not always faster — overhead for small lists
- ⚠ Thread unsafe if shared mutable list is modified without sync — use `collect`, not `forEach + add`
- ⚠ Uses the common pool — can block other parallel tasks — use custom `ForkJoinPool`
- ⚠ Order is not guaranteed — use `forEachOrdered()` if order is needed
- ⚠ `NullPointerException`s are harder to debug in parallel

JAVA 8 TO 21 NEW FEATURES

13/20
13.

13. JAVA 8 TO 21 - INTERVIEW TRICKY QUESTIONS PART 1

Interviewer (Q1):

Difference Stream vs parallelStream?
Why not use Optional as field?

Interviewer (Q2):

Why interface default methods conflict = diamond problem?

Interviewer (Q3):

var limitations?

Candidate (Q1)

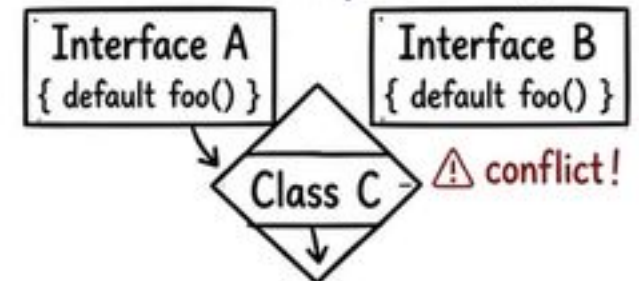
Answer:

- Stream vs parallelStream:
Stream = sequential, single thread, order preserved, simple.
- parallelStream = parallel, uses ForkJoinPool common pool, multi-thread, faster for large data but overhead, thread-safety issues, non-deterministic order.
- Optional should NOT be field or method param - only return type serialization issues, extra wrapper object, anti-pattern. Use Optional for return type of find/search methods.

```
Optional<User> findById(id)
```

Candidate (Q2) – Answer:

- If Class C implements Interface A and B both having default method with same signature -> compiler error.
- Solution: 1) overriding class wins if class provides implementation, 2) else implementing class MUST override and explicitly choose: InterfaceName.super.method{



```
class C implements A, B {
    public void foo() {
        A.super.foo(); // choose A
    }
}
```

Candidate (Q3) – Answer:

- Cannot use for fields, method params, without initializer, with null, lambda params without type.
- Must have initializer; infer locally only.

```
var x; // INVALID
var x = null; // INVALID
class { var field = 10; } // INVALID
```

@CodeEra.Tech

Save & Share for Interview Prep!

JAVA 8 TO 21 NEW FEATURES

14/20

14

JAVA 8 TO 21 - INTERVIEW TRICKY PART 2 - RECORD vs CLASS LOMBOK

Interviewer

Record vs Lombok
@Data? • Virtual Thread
pinning?

Candidate

detailed answer:

- Record vs Lombok: Record = built-in JDK, immutable, compact, no dependency. Lombok = generates bytecode, needs plugin, mutable.
- Record cannot extend, Lombok can.
- Record auto equal/hashCode based on ALL components, Lombok configurable.
- Prefer Record for DTOs in Java 16+.
- Virtual Thread pinning: when virtual thread executes synchronized block or native method it pins carrier platform thread, blocking it.
→ avoid synchronized, use ReentrantLock instead.
- Check with: -Djdk.tracePinnedThreads
- Scoped Values (Java 21 preview) = replacement for ThreadLocal, works with virtual threads.

Record vs Lombok — Comparison Table

Feature	Record	Lombok @Data
Immutability	Immutable by design	Mutable by default
Boilerplate	None, auto-generated	Generated via @Data annotation
Inheritance	Cannot extend (sealed/final)	Can extend other classes
Dependency	Built-in JDK, no lib	Requires Lombok dependency + plugin

Virtual Thread Pinning — What Happens?



Avoid: synchronized
synchronized
void critical() {
 // pins virtual thread
 sharedResource.update();
}

Tip: Pinning blocks
the carrier thread,
reducing
concurrency!

Prefer: ReentrantLock
ReentrantLock
lock = new ReentrantLock();
lock.lock();
try {
 sharedResource.update();
} finally {
 lock.unlock();
}

JAVA 8 TO 21 NEW FEATURES

15/20

15

15. JAVA 8 TO 21 - REAL PROD USE CASES - WHERE TO USE WHAT

Interviewer

When to use Streams? Records?
Virtual Threads in prod?

Candidate

Here are REAL production
use cases.

Box 1: Streams

- data transformation, filtering, mapping, grouping

```
list.stream().filter().map().collect()
```

NOT FOR:

- simple loop with side effects, performance critical tight loops

Box 2: Records – FOR:

- DTOs, API Request Response, Kafka messages
- cache keys, config holders

```
record OrderCreatedEvent(Long orderId,  
BigDecimal amount) {}
```

- compact, immutable, auto equals & hashCode

Use cases:

 REST API

 DTO

 Kafka

Box 3: Virtual Threads FOR:

- IO bound high concurrency: REST API server handling 100k requests
- DB calls, external API calls

```
Executors.newVirtualThreadPerTaskExecutor()
```

Spring Boot 3.2 config:

```
spring.threads.virtual.enabled=true
```

- Tomcat will now use virtual threads

NOT FOR: - CPU bound computation  100k concurrent
- use platform threads

Box 4: Pattern Matching switch FOR:

- cleaner strategy pattern, replace if-else instespoove chain
- cleaner, more readable code



```
switch (shape) {  
case Circle c -> ...  
case Rectangle r -> ...  
}
```


JAVA 8 TO 21 NEW FEATURES 16/20

JAVA 8 TO 21 - MIGRATION GUIDE - FROM JAVA 8 TO 17 → 21

Interviewer (red):
How to migrate from
Java 8 to 17 to 21?

Candidate (blue):
Migration steps:

1. Update build → Maven/Gradle: set Java version to 17 or 21
2. Replace Date/Calendar with **java.time** (LocalDate, LocalDateTime, Instant)
3. Replace anonymous classes with lambda expressions where possible
4. Use **var** for local variable readability where type is clear
5. Replace POJOs with Records for DTOs
6. Use List.of / Set.of / Map.of instead of Collections.unmodifiable...
7. Replace StringBuffer with text blocks `"""` for JSON/multiline strings
8. Enable new HttpClient (java.net.http.HttpClient) for modern HTTP calls
9. For Spring Boot 3 → requires Java 17+, switch to Jakarta EE: use `japx.*`
10. Enable virtual threads → set `spring.threads.virtual.enabled=true`

Common breaking changes:

- Nashorn JavaScript engine removed → use GraalVM JS if JS needed
- Security Manager is deprecated → removed in Java 21
- Stronger module encapsulation → add `--add-opens` if reflective access error occurs

Migration Checklist — 10 Steps ✓

- | | |
|---|---|
| ☒ Update build & Java version | ☒ Use immutable collections (of) |
| ☒ Switch to java.time APIs | ☒ Use Text Blocks for JSON |
| ☒ Refactor to lambdas | ☒ Migrate to HttpClient |
| ☒ Use var for local variables | ☒ Update Spring Boot 3 + Jakarta EE |
| ☒ Convert DTOs to Records | ☒ Enable virtual threads (config) |

Java Version Timeline — LTS Releases



Tip: Test on each version, run `jdeps` to find module issues, update dependencies for Jakarta.

JAVA 8 TO 21 NEW FEATURES

17/20
17.

17. JAVA 8 TO 21 - PERFORMANCE TIPS - STREAMS & RECORDS



Interviewer (red):
Performance tips for
Streams, Records,
Virtual Threads?

Candidate (blue):

SECTION 1: STREAMS — PERFORMANCE

- Avoid boxed streams: Use `IntStream/LongStream/DoubleStream` for primitives
- Use primitive streams: `mapToInt`, `mapToLong` to avoid boxing
- Avoid parallel for small data: overhead > benefit if <10k elements
- Reuse: streams cannot be reused after terminal operation — create new
- Use `collect()` not `forEach()` with external list (thread-safe + faster)

Before:

`Stream-Inteder>.sum()` —
slow due to boxing

After:

`IntStream.range(0, 100).sum()` — faster!

SECTION 2: RECORDS — PERFORMANCE

- Records are faster than class: `equals/hashCode` optimized, compact
- Immutable good for GC: less allocation, better cache locality
- Use as DTO, cache key, Map key

Code:

```
record User(int id, String name) {}
```

vs class with boilerplate

SECTION 3: VIRTUAL THREADS (Java 21) — PERFORMANCE

- Create 1M virtual threads cheap: ~1KB vs 1MB platform thread
- Don't pool them: create per task → `newThreadPerTaskExecutor`
- Avoid `ThreadLocal` heavy use: use `ScopedValue` instead (Java 20+)
- Use `ReentrantLock` not synchronized to avoid pinning carrier thread
- GC: ZGC / Shenandoah low latency for Java 17+ with virtual threads

Code:

```
Executors.newVirtualThreadPerTaskExecutor() vs  
fixedThreadPool
```


JAVA 8 TO 21 NEW FEATURES

18/20 title

18

JAVA 8 TO 21 - CHEAT SHEET ALL FEATURES TIMELINE

Interviewer:

Quick revision all Java versions?

Candidate:

Timeline:

Java 8

- Lambdas
- Streams
- Optional
- DateTime
- CompletableFuture

Java 9

- Modules
- jshell
- List.of / Set.of / Map.of
- takeWhile / dropWhile
- private interface methods

Java 10

- var local

Java 11

- String isBlank / lines / strip / repeat
- Files readString / writeString
- var in lambda
- HttpClient

Java 12

- Switch expressions (preview)
- Collectors.teeing
- String indent

Java 13

- Text blocks
- yield

Java 14

- Records (preview)
- Switch expressions (final)
- insespeep pattern matching

Java 15

- Sealed (preview)
- Text blocks (final)

Java 21

- Virtual threads (final)
- Sequenced Collections
- String templates (preview)
- Pattern matching switch (final)
- Scoped Values

Java 17

- Sealed (final)
- LTS
- Pattern matching switch (preview)

Java 18

- UTF-8 default
- Simple web server

Java 19

- Virtual threads (preview)
- Record patterns

JAVA 8 TO 21 NEW FEATURES

19/20

19.

19. JAVA 8 TO 21 - CODE CHALLENGE - BEFORE AFTER REFACTOR



Interviewer (red):
Refactor Java 7 code
to Java 21 modern?



Candidate (blue):
Before vs After

BEFORE - Java 7 - VERBOSE (60+ lines!)

```
class Person {
    private String name;
    private int age;
    // constructor
    // getters
    // equals() hashCode()
    // toString() - 60 lines!
    for(Person p : persons) {
        if(p.getAge() > 18) {
            list.add(p);
        }
    }
    Date date = new Date();
    new Thread(new Runnable() {
        public void run() task();
    }).start();
    if(user != null && user.getAddress() != null
        && user.getAddress().getCity() != null) { ... }
    String s = "Hello " + name + "?";
```

AFTER - Java 21 - MODERN (15 lines!) Clean!

```
record Person(String name, int age) {}

List<Person> adults = persons.stream()
    .filter(p -> p.age() > 18)
    .toList();
LocalDate.now()
Thread.startVirtualThread(>) -> task(>);
Optional.ofNullable(user)
    .map(User::getAddress)
    .map(Address::getCity)
    .orElse("Unknown");
String s = STR."Hello {name}!!!";
```

=> **80% LESS CODE**

100 lines → 15 lines ✓

More Readable!

QUICK WIN: Records + Streams + toList() + Virtual Threads
+ Optional + String Templates + LocalDate = Modern Java!

JAVA 8 TO 21 NEW FEATURES

20/20
20

FINAL HR & HOW TO ANSWER & CTA

Interviewer (red): Tell me about modern Java feature you used in prod?

Candidate (blue) STAR:

- Situation: Prod API 5 sec blocking threads, 200 limit
- Task: Migrate to Java 21 Virtual Threads
- Action:
 - Enabled `spring.threads.virtual.enabled=true`
 - Replaced `synchronized` with `ReentrantLock` to avoid pinning
 - Used `Records` for DTOs — reduced 60% boilerplate
 - Used pattern matching switch for strategy, clean code
- Result: Throughput 200 → 5000 concurrent • 25x faster
Latency 5 sec → 200ms • GC improved

HR Tips — How to Answer Well

1. Never say “I just know Java 8” — show before/after code
2. Mention virtual threads + pinning risk (`synchronized` blocks can pin)
3. Mention `sealed`, `records`, pattern matching as modern Java features
4. Say: “Java 17 LTS and Java 21 LTS are the new Java 8”

SAVE THIS — 20 SLIDES JAVA 8 to 21 COMPLETE GUIDE

Follow @CodeEra.Tech for more Java interview tips & guides